6.3 P2P 네트워킹 및 통신 분석

개요

Composable Finance 노드는 Substrate의 네트워킹 스택을 기반으로 하여 libp2p 프로토콜을 사용한 P2P 네트워킹을 구현합니다. 이 문서에서는 libp2p 기반 네트워크 스택 구성, 피어 발견 및 연결 관리 메커니즘, 그리고 블록 동기화 및 트랜잭션 전파 과정을 상세히 분석합니다.

6.3.1 libp2p 기반 네트워크 스택 구성

네트워크 설정 초기화

노드의 네트워크 구성은 `sc_service::build_network` 함수를 통해 초기화됩니다:

```
// service.rs에서
let  net_config = sc_network::config::FullNetworkConfiguration::new(&parachain_c


let (network, system_rpc_tx, tx_handler_controller, start_network, sync_service)
sc_service::build_network(sc_service::BuildNetworkParams {
config: &parachain_config,
net_config,
client: client.clone(),
transaction_pool: transaction_pool.clone(),
spawn_handle,
import_queue: params.import_queue,
```

```
warp_sync_params,
block_announce_validator_builder: Some(Box::new(|_| {
Box::new(block_announce_validator)
})),
})?;
```

네트워크 구성 요소

네트워크 스택은 다음과 같은 주요 구성 요소를 포함합니다:

1. **네트워크 서비스**: P2P 통신의 핵심 서비스
2. **시스템 RPC 송신자**: 시스템 레벨 RPC 통신
3. **트랜잭션 핸들러 컨트롤러**: 트랜잭션 전파 관리
4. **네트워크 시작 핸들**: 네트워크 서비스 시작 제어
5. **동기화 서비스**: 블록 동기화 관리

기본 포트 설정

릴레이체인 CLI에서 정의된 기본 포트 설정:

```
impl DefaultConfigurationValues for RelayChainCli {
fn p2p_listen_port() -> u16 {
30334 // 릴레이체인 P2P 포트
}

fn prometheus_listen_port() -> u16 {
```

```
9616  // 프로메테우스 메트릭 포트
}

fn  rpc_listen_port() -> u16 {
sc_cli::RPC_DEFAULT_PORT // 기본 RPC 포트 (9944)
}
}
```

파라체인 노드는 일반적으로 다른 포트를 사용합니다:

- P2P 포트: 30333 (기본값)
- RPC 포트: 9944
- WebSocket 포트: 9944
- 프로메테우스 포트: 9615

6.3.2 피어 발견 및 연결 관리 메커니즘

블록 공지 검증자

파라체인은 특별한 블록 공지 검증자를 사용합니다:

```
let  block_announce_validator =
BlockAnnounceValidator::new(relay_chain_interface.clone(), para_id);
```

이 검증자는 다음 기능을 수행합니다:

- **파라체인 블록 검증**: 수신된 블록 공지가 유효한 파라체인 블록인지 검증

- **릴레이체인 연동**: 릴레이체인 상태와 일치하는지 확인
- **파라체인 ID 검증**: 올바른 파라체인에서 온 블록인지 확인

피어 발견 프로토콜

libp2p는 다음과 같은 피어 발견 메커니즘을 사용합니다:

1. **mDNS (Multicast DNS)**: 로컬 네트워크에서 피어 자동 발견
2. **Kademlia DHT**: 분산 해시 테이블을 통한 피어 발견
3. **부트스트랩 노드**: 체인 스펙에 정의된 초기 연결 노드들
4. **피어 교환**: 연결된 피어로부터 다른 피어 정보 획득

연결 관리

네트워크 서비스는 다음과 같은 연결 관리 기능을 제공합니다:

- **최대 연결 수 제한**: 리소스 관리를 위한 연결 수 제한
- **연결 품질 평가**: 피어의 응답성과 신뢰성 평가
- **자동 재연결**: 연결 끊김 시 자동 재연결 시도
- **밴 관리**: 악의적이거나 문제가 있는 피어 차단

6.3.3 블록 동기화 및 트랜잭션 전파 과정

동기화 서비스

동기화 서비스는 블록체인 상태 동기화를 관리합니다:

```
let announce_block = {
let sync_service = sync_service.clone();
Arc::new(move |hash, data| sync_service.announce_block(hash, data))
```

```
};
```

블록 임포트 큐

블록 임포트는 전용 큐를 통해 처리됩니다:

```
pub fn parachain_build_import_queue<RuntimeApi, Executor>(
backend: Arc<FullBackend>,
client: Arc<FullClient<RuntimeApi, Executor>>,
config: &Configuration,
telemetry: Option<TelemetryHandle>,
task_manager: &TaskManager,
) -> Result<(
sc_consensus::DefaultImportQueue<OpaqueBlock, FullClient<RuntimeApi, Executor>>,
ParachainBlockImport<OpaqueBlock, Arc<FullClient<RuntimeApi, Executor>>, FullBac
), sc_service::Error>
```

임포트 큐는 다음 기능을 수행합니다:

1. **블록 검증**: 수신된 블록의 유효성 검증
2. **순서 보장**: 블록이 올바른 순서로 임포트되도록 보장
3. **병렬 처리**: 여러 블록을 병렬로 검증 및 임포트
4. **오류 처리**: 잘못된 블록에 대한 적절한 오류 처리

AURA 기반 임포트 큐 설정

파라체인은 AURA 컨센서스에 특화된 임포트 큐를 사용합니다:

```
cumulus_client_consensus_aura::import_queue::<
sp_consensus_aura::sr25519::AuthorityPair,
_,
_,
_,
_,
_,
>(cumulus_client_consensus_aura::ImportQueueParams {
block_import: block_import.clone(),
client,
create_inherent_data_providers: move |_, _| async  move {
let  time = sp_timestamp::InherentDataProvider::from_system_time();
let  slot = sp_consensus_aura::inherents::InherentDataProvider::from_timestamp_a
*time,
slot_duration,
);
Ok((slot, time))
},
registry: config.prometheus_registry(),
spawner: &task_manager.spawn_essential_handle(),
telemetry,
```

```
})
```

워프 동기화

빠른 동기화를 위해 워프 동기화를 지원합니다:

```
let warp_sync_params = match parachain_config.network.sync_mode {
sc_network::config::SyncMode::Warp => {
let target_block = warp_sync_get::<OpaqueBlock, _>(
para_id,
relay_chain_interface.clone(),
spawn_handle.clone(),
);
Some(WarpSyncParams::WaitForTarget(target_block))
},
_ => None,
};
```

워프 동기화 과정:

1. **타겟 블록 결정**: 릴레이체인의 최종화된 블록을 기준으로 타겟 설정
2. **상태 다운로드**: 타겟 블록의 상태를 직접 다운로드
3. **검증**: 다운로드된 상태의 유효성 검증
4. **적용**: 검증된 상태를 로컬 데이터베이스에 적용

6.3.4 트랜잭션 전파 메커니즘

트랜잭션 풀 통합

네트워크 서비스는 트랜잭션 풀과 통합되어 트랜잭션을 전파합니다:

```
sc_service::build_network(sc_service::BuildNetworkParams {
// ...
transaction_pool: transaction_pool.clone(),
// ...
})
```

트랜잭션 전파 과정

1. **트랜잭션 수신**: RPC 또는 다른 피어로부터 트랜잭션 수신
2. **유효성 검증**: 트랜잭션의 기본 유효성 검증
3. **풀 추가**: 유효한 트랜잭션을 트랜잭션 풀에 추가
4. **피어 전파**: 연결된 피어들에게 트랜잭션 전파
5. **중복 제거**: 이미 알고 있는 트랜잭션 중복 전파 방지

가십 프로토콜

트랜잭션과 블록 공지는 가십 프로토콜을 통해 전파됩니다:

- **확률적 전파**: 모든 피어가 아닌 일부 피어에게만 전파
- **중복 감지**: 이미 전파된 메시지 중복 전파 방지
- **TTL (Time To Live)**: 메시지의 생존 시간 제한
- **백프레셔**: 네트워크 혼잡 시 전파 속도 조절

6.3.5 오프체인 워커 네트워킹

오프체인 워커 활성화

오프체인 워커가 활성화된 경우 추가 네트워킹 기능을 제공합니다:

```
if  parachain_config.offchain_worker.enabled {
sc_service::build_offchain_workers(
&parachain_config,
task_manager.spawn_handle(),
client.clone(),
network.clone(),
);
}
```

오프체인 네트워킹 기능

오프체인 워커는 다음과 같은 네트워킹 기능을 사용합니다:

1. **HTTP 요청**: 외부 API 호출
2. **DNS 조회**: 도메인 이름 해석
3. **TCP/UDP 소켓**: 직접적인 네트워크 통신
4. **웹소켓 연결**: 실시간 데이터 스트림

이러한 기능은 Oracle 팰릿과 같이 외부 데이터가 필요한 기능에 필수적입니다.

6.3.6 네트워크 보안 및 인증

피어 인증

libp2p는 다음과 같은 인증 메커니즘을 사용합니다:

1. **Ed25519 키 쌍**: 노드 신원 확인
2. **Noise 프로토콜**: 연결 암호화 및 인증
3. **피어 ID**: 공개키 기반 고유 식별자
4. **서명 검증**: 메시지 무결성 및 출처 확인

네트워크 보안 기능

- **연결 암호화**: 모든 P2P 연결 암호화
- **메시지 인증**: 메시지 출처 및 무결성 검증
- **DDoS 방어**: 연결 수 제한 및 속도 제한
- **악의적 피어 차단**: 문제가 있는 피어 자동 차단

6.3.7 네트워크 모니터링 및 텔레메트리

네트워크 메트릭

프로메테우스를 통해 다음과 같은 네트워크 메트릭을 수집합니다:

- **연결된 피어 수**: 현재 연결된 피어의 수
- **대역폭 사용량**: 송수신 데이터량
- **블록 동기화 속도**: 블록 다운로드 및 검증 속도
- **트랜잭션 전파 지연**: 트랜잭션 전파에 걸리는 시간

텔레메트리 데이터

텔레메트리 시스템을 통해 네트워크 상태를 모니터링합니다:

```
let  telemetry = config
```

```
.telemetry_endpoints
.clone()
.filter(|x| !x.is_empty())
.map(|endpoints| -> Result<_, sc_telemetry::Error> {
let  worker = TelemetryWorker::new(16)?;
let  telemetry = worker.handle().new_telemetry(endpoints);
Ok((worker, telemetry))
})
.transpose()?;
```

6.3.8 네트워크 시작 및 종료

네트워크 서비스 시작

네트워크 서비스는 모든 구성 요소가 준비된 후 시작됩니다:

```
start_network.start_network();
```

정상 종료

노드 종료 시 네트워크 연결을 정상적으로 종료합니다:

1. **새 연결 중단**: 새로운 피어 연결 중단
2. **기존 연결 종료**: 연결된 피어들에게 종료 신호 전송
3. **리소스 정리**: 네트워크 관련 리소스 정리
4. **상태 저장**: 필요한 네트워크 상태 정보 저장

결론

Composable Finance의 P2P 네트워킹 시스템은 libp2p를 기반으로 한 견고하고 확장 가능한 네트워크 아키텍처를 제공합니다. 효율적인 피어 발견, 안전한 통신, 빠른 블록 동기화, 그리고 신뢰할 수 있는 트랜잭션 전파를 통해 분산 네트워크에서의 안정적인 운영을 보장합니다. 특히 파라체인 환경에 최적화된 블록 검증과 릴레이체인과의 통합을 통해 Polkadot 생태계에서의 원활한 상호운용성을 제공합니다.